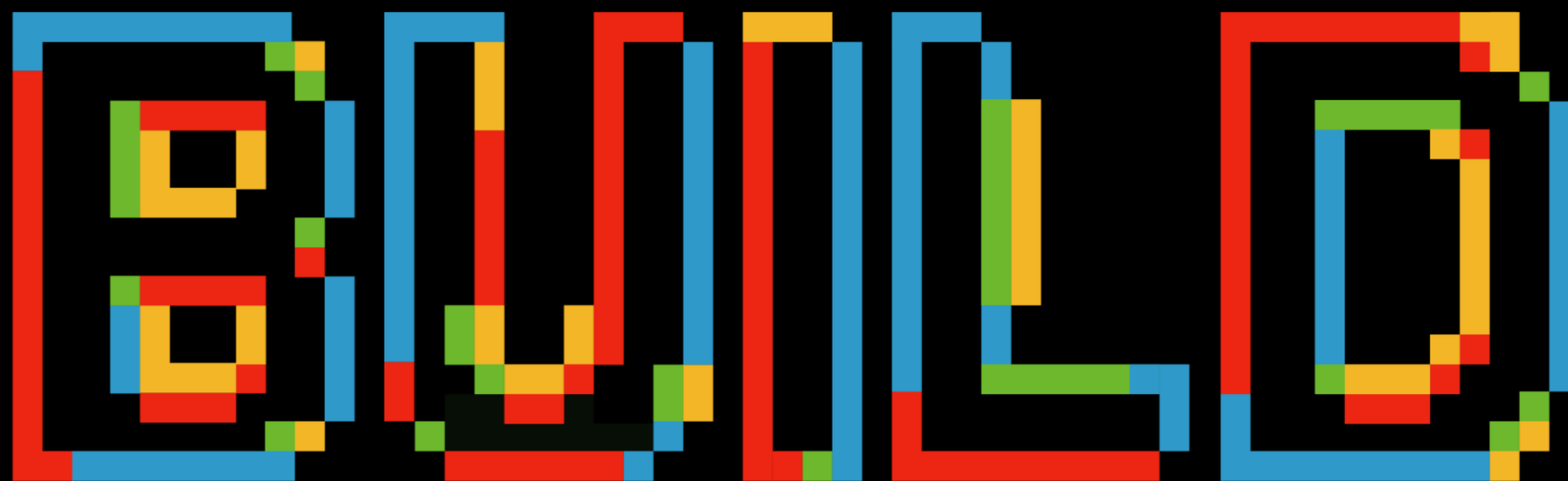
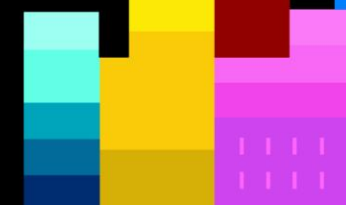
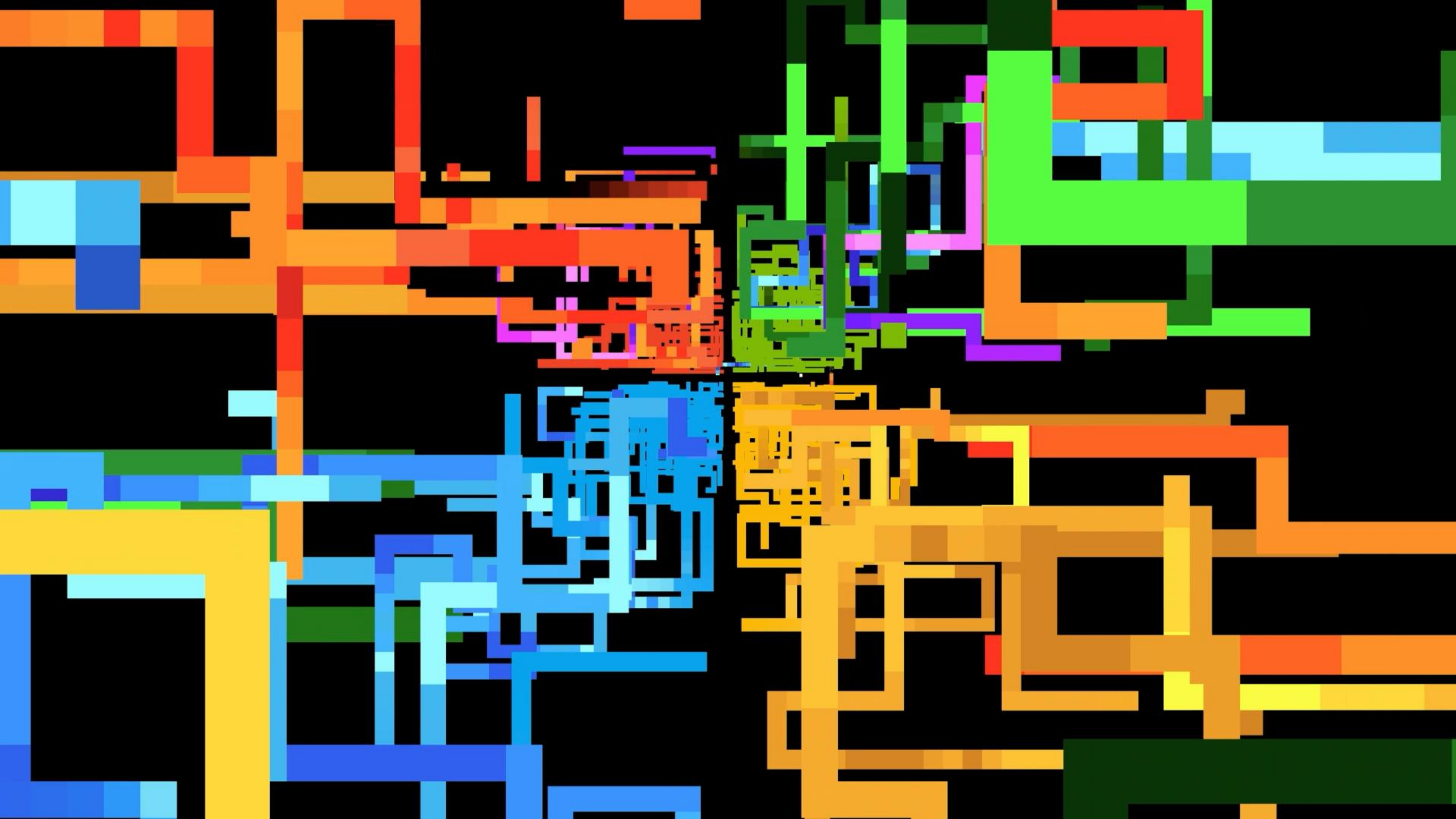




MICROSOFT BUILD



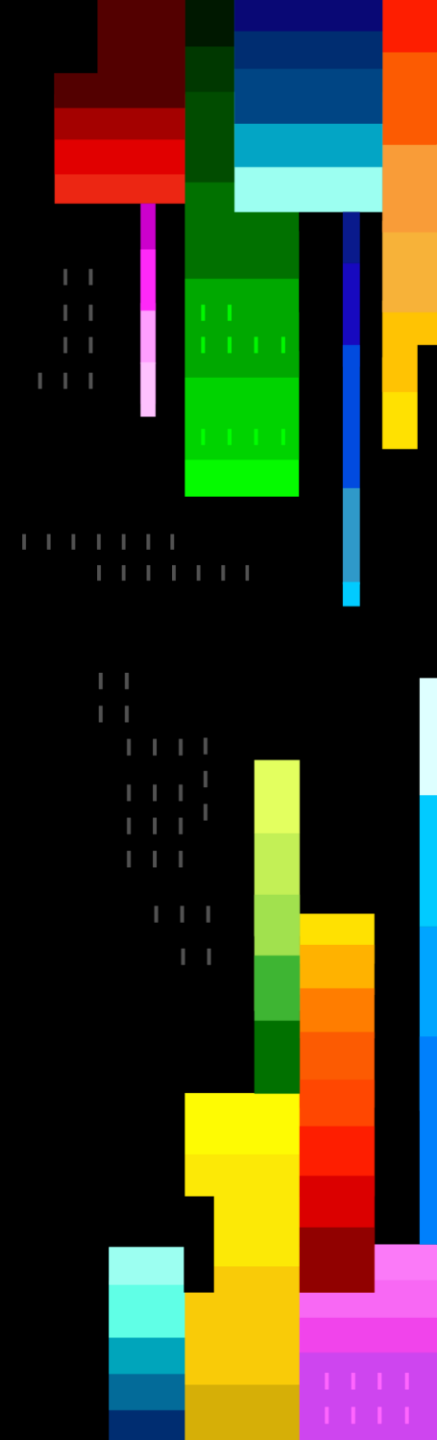




From zero to deployed on Azure with AI agents



MD Shahriyar Al Mustakim Mitul
Lead, Azure Tech Group Bangladesh



Agenda

- Introduction to AI-Powered Development with GitHub Copilot CLI and Coding Agents
- Building an Application from an Empty Terminal Using AI Agents
- Application Scaffolding and Code Generation with Natural Language
- Debugging and Iterating with AI-Assisted Development
- Azure Skills Overview
- Provisioning Azure Resources with Natural Language
- Deploying Applications to Azure Using Azure Developer CLI (azd)
- Monitoring and Managing Applications in Azure
- AI-Native Development Workflows and Best Practices
- Responsible AI and Secure Application Development
- Production-Ready Deployment Strategies
- Q&A and Next Steps



Run the demo with me



Student Access to Labs:
5F740095E196477F



AI can deploy your app to Azure in 5 minutes. But should you trust what it built? In this lab, you'll use GitHub Copilot CLI with Azure skills to deploy a live Container App — a Python Flask application that browses a LEGO set catalog backed by Azure Cosmos DB — along with a Function App that upserts new LEGO sets into Cosmos DB. Then, you'll put on your architect hat and evaluate the AI's decisions. You'll review the generated Bicep files, identify what's missing for production readiness, direct the AI to harden the deployment, intentionally break the app, and run a full forensic investigation — all without opening the Azure Portal.

This lab teaches you **how to use AI to deploy an application, then verify and improve what the AI created**

AI builds and deploys the application

Instead of manually creating Azure resources, you ask **GitHub Copilot CLI**.
It automatically:

- Creates a **Python Flask** web application.
- Deploys it to **Azure Container Apps**.
- Creates an **Azure Cosmos DB** database to store LEGO sets.
- Creates an **Azure Function App** that can add (upsert) new LEGO sets into the database.

Within a few minutes, you have a working application.

Test the application

The application lets you:

- Browse LEGO sets.

- Add or update LEGO sets through the Azure Function.

- Store everything in Cosmos DB.

- So you confirm the deployment actually works.

Inspect what the AI created

Just because AI is deployed, it doesn't mean it's perfect.
You review the generated **Bicep** infrastructure files (Infrastructure-as-Code).
You ask questions like:

- Is security configured properly?
- Are secrets stored safely?
- Is monitoring enabled?
- Is logging configured?
- Is the deployment production-ready?

Improve the deployment

You then tell the AI things like:

- "Enable managed identity."
- "Store secrets in Key Vault."
- "Turn on Application Insights."
- "Restrict public access."
- "Add health checks."

The AI updates the infrastructure accordingly.

Break the application on purpose

To learn troubleshooting, you intentionally introduce problems.

For example:

- Wrong database connection string
- Deleted environment variable
- Broken Function App
- Incorrect permissions

The app stops working.

Investigate the failure

Instead of opening the Azure Portal, you diagnose everything from the terminal. You use AI together with Azure CLI to:

- View logs
- Check deployments
- Inspect Container Apps
- Examine Function App logs
- Verify Cosmos DB connectivity
- Find the root cause

GitHub Repo



Follow the guide



Follow me on LinkedIn/X



LinkedIn



X

Submit your project here



<https://codex-community-meetup-dhaka.devpost.com/>



Thank you!